



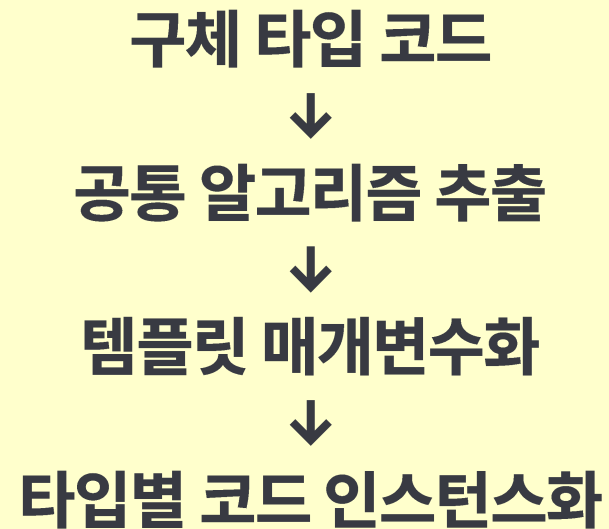
10. 템플릿과 제네릭 프로그래밍

Heesung Oh

<https://github.com/3dapi>



- 제네릭 프로그래밍
- 함수 템플릿
- 클래스 템플릿
- 템플릿과 헤더 파일
- 템플릿 특수화
- 가변 인수 템플릿
- Concepts와 제약 조건
- 타입 특성과 조건부 코드
- 전달 참조와 완벽 전달





```
template <typename T>
T Clamp(T value, T minimum, T maximum)
{
    if (value < minimum)
    {
        return minimum;
    }
    else if (maximum < value)
    {
        return maximum;
    }
    return value;
}
```

```
int score = Clamp(1200, 0, 999);
float speed = Clamp(12.5f, 0.0f, 10.0f);
```

- 코드 모양만 같다고 일반화하지 않음
- 변경 이유와 의미가 같을 때 일반화

호출	추론 결과
Clamp(1200, 0, 999)	T = int
Clamp(12.5f, 0.0f, 10.0f)	T = float



- 템플릿은 구체 타입 이름에서 독립적
- 알고리즘이 사용하는 연산에서는 독립적이지 않음
- Clamp()는 비교 연산과 반환 가능성을 요구

```
class Level
{
public:
    explicit Level(int value) : value(value)
    {
    }
    friend bool operator<(const Level& left, const Level& right)
    {
        return left.value < right.value;
    }
}
```

```
private:
    int value;
};
```

```
Level current(120);
Level minimum(1);
Level maximum(99);
Level result = Clamp(current, minimum, maximum);
```

- 문법적으로 가능한 연산과 도메인 의미는 별도로 판단



10.1.3 객체지향 다형성과 정적 다형성

기준	객체지향 다형성	정적 다형성
구현 선택 시점	실행 시간	컴파일 시점
공통 형식	기반 클래스와 가상 함수	템플릿과 요구 조건
구현 교체	기반 포인터 교체	구체 템플릿 타입 변경
호출 비용	간접 호출 가능	인라인 최적화 가능
코드 크기	호출 코드 공유	타입별 코드 증가 가능
적합한 상황	실행 중 객체 종류 관리	타입별 최적화와 일반화

```
class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;
    virtual int CalculateDamage() const = 0;
};
```

```
template <typename AttackPolicy>
class AttackProcessor
{
public:
    int Execute() const
    {
        return policy.CalculateDamage();
    }
private:
    AttackPolicy policy;
};
```



- 함수의 타입이나 값을 템플릿 매개변수로 분리
- 함수 템플릿 자체는 특정 타입의 함수가 아님
- 함수를 생성하기 위한 틀

```
template <typename T>  
T Square(T value)  
{  
    return value * value;  
}
```

```
int first = Square(10);  
double second = Square(3.5);
```

- typename 대신 class도 사용할 수 있음

표기	의미
template	템플릿 선언 시작
typename T	타입 템플릿 매개변수
T Square(T value)	T 타입의 값을 받아 T 반환



```
template <typename T>  
T GetMin(T left, T right)  
{  
    return right < left ? right : left;  
}
```

```
int first = GetMin(10, 20);  
double second = GetMin(3.5, 1.2);
```

```
// auto value = GetMin(10, 2.5); // 오류
```

- 하나의 T가 여러 매개변수에 사용되면 같은 타입 필요
- 암시적 변환은 템플릿 인수 추론 뒤에 적용

호출	추론
GetMin(10, 20)	T = int
GetMin(3.5, 1.2)	T = double
GetMin(10, 2.5)	T를 하나로 결정할 수 없음



10.2.2 값 매개변수와 참조 매개변수 추론

```
#include <type_traits>
```

```
template <typename T>  
void InspectValue(T value)  
{  
    static_assert(std::is_same_v<T, int>);  
    value = 20;  
}
```

```
const int number = 10;  
InspectValue(number);
```

```
template <typename T>  
void InspectReference(T& value)  
{  
    static_assert(std::is_same_v<T, const int>);  
}  
  
const int number = 10;  
InspectReference(number);
```

매개변수	추론 특징
T value	최상위 const와 참조성 제거
T& value	참조 대상의 const 보존
const T& value	복사 없이 읽기 전용 전달



10.2.2 배열 크기 추론

```
#include <cstddef>

template <typename T, std::size_t Size>
constexpr std::size_t ArraySize(const T (&)[Size]) noexcept
{
    return Size;
}

int values[5]{};
static_assert(ArraySize(values) == 5);
```

<일반>
int[5]
↓ 값 매개변수
int*

<템플릿>
int[5]
↓ 배열 참조 매개변수
T = int, Size = 5

매개변수 형태	배열 처리
T value	배열이 포인터로 변환됨
T& value	배열 타입 보존
const T (&)[Size]	원소 타입과 크기 보존





```
template <typename T>  
T Convert(int value)  
{  
    return static_cast<T>(value);  
}
```

// <double> : 명시적 템플릿 인수

// 전달된 인수(10) 으로 반환 타입 T 추론 불가. <double> 사용

```
double realValue = Convert<double>(10);
```

```
template <typename Result, typename Left, typename Right>  
Result AddAs(const Left& left, const Right& right)  
{  
    return static_cast<Result>(left + right);  
}
```

```
double result = AddAs<double>(10, 2.5f);
```

```
int result = GetMin<int>(3.8, 2.2);
```

- 명시적 인수는 강제 변환을 만들 수 있음
- 컴파일 성공과 의도한 결과는 별도 확인



10.2.4 함수 오버로딩과 템플릿

```
#include <iostream>
```

```
void Print(int value)
```

```
{  
    std::cout << "int: " << value << '\n';  
}
```

```
template <typename T>
```

```
void Print(const T& value)
```

```
{  
    std::cout << "template: " << value << '\n';  
}
```

```
Print(10); // 일반 함수 Print(int)
```

```
Print(3.5); // 함수 템플릿 Print<double>
```

```
Print(10); // 일반 함수 Print(int)
```

```
Print(3.5); // 함수 템플릿 Print<double>
```

- 같은 수준의 변환으로 호출 될 때 일반 함수가 우선

```
void Show(long value)
```

```
{  
    std::cout << "long: " << value << '\n';  
}
```

```
template <typename T>
```

```
void Show(T value)
```

```
{  
    std::cout << "template: " << value << '\n';  
}
```

```
Show(10); // 함수 템플릿 Show<int>
```

- 템플릿 후보가 더 좋은 변환을 제공하면 템플릿이 우선 선택



10.2.5 템플릿 인스턴스화

- 함수 템플릿 정의만으로 모든 함수가 생성되지 않음
- 실제 사용된 템플릿 인수 조합에 대해 코드 생성
- 이 과정을 인스턴스화 라고 함

```
template <typename T>  
T Multiply(T left, T right)  
{  
    return left * right;  
}
```

```
int integerResult = Multiply(3, 4);  
double realResult = Multiply(1.5, 2.0);
```

```
template <typename T>  
void PrintName(const T& value)  
{  
    std::cout << value.GetName() << '\n';  
}
```

```
Multiply<int>(int, int)  
Multiply<double>(double, double)
```

- 템플릿 인수에 의존하는 표현은 인스턴스화 시점에 검사



- 데이터 멤버와 멤버 함수의 타입이나 값을 템플릿 매개변수로 분리
- 저장할 원소 타입과 크기를 일반화 가능
- 각 템플릿 인수 조합은 서로 다른 구체 타입

```
#include <cstdint>
#include <stdexcept>

template <typename T, std::size_t Size>
class FixedArray
{
public:
    T& At(std::size_t index);
    const T& At(std::size_t index) const;

    constexpr std::size_t SizeValue() const noexcept
    {
        return Size;
    }

private:
    static_assert(Size > 0);
    T values[Size]{};
};
```

```
FixedArray<int, 4> scores;
FixedArray<double, 8> weights;
```



10.3.2 멤버 함수 정의

```
template <typename T>
class Box
{
public:
    explicit Box(const T& value);
    const T& GetValue() const;
    void SetValue(const T& value);

private:
    T value;
};
```

```
template <typename T>
Box<T>::Box(const T& value) : value(value)
{
}

template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}

template <typename T>
void Box<T>::SetValue(const T& value)
{
    this->value = value;
}
```

- 클래스 밖 정의에는 `template <...>` 필요
- 클래스 범위에는 `Box<T>::` 사용



- 템플릿은 호출 지점에서 실제 타입에 맞는 코드가 필요
- 컴파일러가 템플릿 정의 전체를 볼 수 있어야 함
- 일반적으로 선언과 정의를 모두 헤더에 둬

```
// Box.h
#pragma once

template <typename T>
class Box
{
public:
    explicit Box(const T& value) : value(value) { }

    const T& GetValue() const
    {
        return value;
    }

private:
    T value;
};
```

사용 파일에서 Box<int> 필요



Box<T> 정의가 보여야 함



Box<int> 인스턴스화 가능



10.3.3 선언과 구현을 나누는 방식

방식	특징
모두 헤더에 정의	가장 일반적
.inl을 헤더에서 포함	파일 분리, 정의는 여전히 보임
명시적 인스턴스화	지원 타입 제한, 코드 생성 통제

```
// Box.h
#pragma once
template <typename T>
class Box
{
public:
    explicit Box(const T& value);
    const T& GetValue() const;

private:
    T value;
};
```

```
#include "Box.inl"
```

```
// Box.inl
template <typename T>
Box<T>::Box(const T& value)
    : value(value)
{
}

template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}
```



- C++17부터 생성자 인수로 클래스 템플릿 인수 추론 가능
- CTAD: class template argument deduction

```
#include <utility>
template <typename T>
class ValueBox
{
public:
    explicit ValueBox(T value)
        : value(std::move(value))
    {
    }

private:
    T value;
};
```

```
ValueBox first(10); // ValueBox<int>
ValueBox second(3.5); // ValueBox<double>
ValueBox name("Knight"); // ValueBox<const char*>

ValueBox(const char*) -> ValueBox<std::string>;
ValueBox text("Knight"); // ValueBox<std::string>
```

- 추론 가이드는 객체 생성 시 템플릿 인수 결정에 사용



10.3.7 비 타입 템플릿 매개변수

```
template <typename T, std::size_t Size>
class StaticBuffer
{
public:
    constexpr std::size_t GetCapacity() const
    noexcept
    {
        return Size;
    }

private:
    T values[Size]{};
};
```

```
StaticBuffer<int, 32> packet;
StaticBuffer<float, 4> position;

constexpr std::size_t PlayerCount = 4;
StaticBuffer<int, PlayerCount> scores;

std::size_t runtimeCount = 8;
// StaticBuffer<int, runtimeCount> values; // 오류
```

특징	설명
값은 컴파일 시점 상수	런타임 값 불가
값이 타입의 일부	크기 다르면 다른 타입
저장 공간	Size 자체는 멤버가 아님



- 기본 템플릿을 대부분의 타입에 적용
- 특정 타입이나 타입 형태에 다른 구현 제공
- 전체 특수화: 모든 템플릿 인수 고정
- 부분 특수화: 일부 타입 패턴에 적용

```
#include <string_view>
template <typename T>
struct TypeName
{
    static constexpr std::string_view Get()
    {
        return "unknown";
    }
};
template <>
struct TypeName<int>
{
    static constexpr std::string_view Get()
    {
        return "int";
    }
};
```

```
static_assert(TypeName<int>::Get() == "int");
static_assert(TypeName<char>::Get() ==
"unknown");
```



```
template <typename T>
struct TypeCategory
{
    static constexpr std::string_view Get()
    {
        return "value";
    }
};
```

```
template <typename T>
struct TypeCategory<T*>
{
    static constexpr std::string_view Get()
    {
        return "pointer";
    }
};
```

- 함수 템플릿은 부분 특수화를 지원하지 않음
- 함수는 오버로딩으로 표현

```
static_assert(TypeCategory<int>::Get() == "value");
static_assert(TypeCategory<int*>::Get() == "pointer");
static_assert(TypeCategory<double*>::Get() == "pointer");
```

```
template <typename T, std::size_t Size>
struct TypeCategory<T[Size]>
{
    static constexpr std::string_view Get()
    {
        return "fixed array";
    }
};
```



10.4.3 함수 템플릿 오버로딩과 특수화

방법	특징
함수 템플릿 전체 특수화	선언 순서와 가시성에 민감
일반 함수 오버로딩	오버로드 후보로 직접 참여
Concept 제약 오버로딩	타입 범주별 선택 표현

```
template <typename T>
void Save(const T& value)
{
    std::cout << value;
}

void Save(const char* value)
{
    std::cout << "'" << value << "'";
}

void Save(const std::string& value)
{
    std::cout << "'" << value << "'";
}
```

```
template <std::integral T>
void Save(T value);
```

```
template <std::floating_point T>
void Save(T value);
```

- 함수 템플릿의 특정 타입 동작은 일반 오버로딩 우선 검토



- 개수가 정해지지 않은 템플릿 인수와 함수 인수를 받음
- 타입 매개변수 팩과 함수 매개변수 팩 사용
- 로그 출력, 객체 생성, 콜백 조합 등에 활용

```
template <typename... Types>
class TypeList
{
};

TypeList<> emptyTypes;
TypeList<int> oneType;
TypeList<int, double, const char*> threeTypes;

template <typename... Args>
void CountArguments(const Args&... args)
{
    std::cout << sizeof...(Args) << '\n';
    std::cout << sizeof...(args) << '\n';
}
```

표현	의미
typename... Args	타입 매개변수 팩
const Args&... args	함수 매개변수 팩
sizeof...(Args)	팩 원소 개수



```
template <typename T>
void PrintOne(const T& value)
{
    std::cout << value << '\n';
}
```

```
#include <initializer_list>
```

```
template <typename... Args>
void PrintLines(const Args&... args)
{
    (void)std::initializer_list<int>
    {
        (PrintOne(args), 0)...
    };
}
```

```
template <typename Function, typename... Args>
auto Call(Function function, const Args&... args)
{
    return function(args...);
}
```

args...



각 원소를 별도 인수

- ...가 붙은 전체 패턴이 반복됨
- 반복 단위가 함수 호출 전체인지 인수 하나인지 구분



- C++17부터 매개변수 팩 원소 사이에 이항 연산자 반복 적용

```
template <typename... Args>  
auto Sum(const Args&... args)  
{  
    return (... + args);  
}  
  
int integerSum = Sum(10, 20, 30);  
double mixedSum = Sum(10, 2.5, 3.5f);
```

형태	문법	개념적 결합
단항 왼쪽 폴드	(... op pack)	((a op b) op c)
단항 오른쪽 폴드	(pack op ...)	(a op (b op c))
이항 왼쪽 폴드	(init op ... op pack)	((init op a) op b)
이항 오른쪽 폴드	(pack op ... op init)	(a op (b op init))

- 빼기와 나누기는 결합 방향에 따라 결과가 달라짐



```
enum class LogLevel
{
    Info,
    Warning,
    Error
};
template <typename... Args>
void Log(LogLevel level, const Args&... args)
{
    std::cout << '[' << ToString(level) << " ] ";
    ((std::cout << args), ...);
    std::cout << '\n';
}
```

```
Log(LogLevel::Info, "Player hp=", 100);
Log(LogLevel::Warning, "Low stamina: ", 12.5f, '%');
Log(LogLevel::Error, "Cannot open file: ",
    "save.dat");
```

장점	주의점
인수 개수와 타입이 유연함	의미가 위치에 의존하면 혼란 가능
로그와 생성자 전달에 유용	전달에는 값 범주 보존 필요
공통 출력 패턴 구성 가능	출력 가능성 제약 필요



10.6 Concepts (C++20)와 제약 조건

- 템플릿이 받을 수 있는 타입 조건을 선언에 표현
- 오류 위치를 호출 지점에 가깝게 만들 수 있음
- 오버로드 후보 제한과 더 구체적인 후보 선택 가능

```
#include <concepts>
template <std::integral T>
T AddInteger(T left, T right)
{
    return left + right;
}

int value = AddInteger(10, 20);
// double real = AddInteger(1.5, 2.5); // 오류
```

제약 없는 템플릿



본문 내부에서 오류 발생 가능

Concept 적용



인터페이스에서 요구 조건 표현





Concept	의미
<code>std::same_as<T, U></code>	두 타입이 같음
<code>std::derived_from<D, B></code>	D가 B의 공개 파생 타입
<code>std::convertible_to<F, T></code>	F에서 T로 변환 가능
<code>std::integral<T></code>	정수형 범주
<code>std::floating_point<T></code>	부동소수점형 범주
<code>std::constructible_from<T, Args...></code>	지정한 인수로 T 생성 가능
<code>std::copyable<T></code>	복사 가능한 값 타입 요구
<code>std::movable<T></code>	이동 가능한 값 타입 요구
<code>std::invocable<F, Args...></code>	지정한 인수로 호출 가능
<code>std::predicate<F, Args...></code>	논리 판정 함수로 사용 가능

```
template <std::floating_point T>
T HalfValue(T value)
{
    return value / static_cast<T>(2);
}
```

- 의미 조건까지 컴파일러가 검증하는 것은 아님



- requires : 템플릿 선언에 논리 제약 추가

```
#include <concepts>

template <typename T>
requires std::integral<T>
T AbsValue(T value)
{
    return value < 0 ? -value : value;
}

template <typename T>
T AbsValue(T value)
requires std::integral<T>
{
    return value < 0 ? -value : value;
}

template <std::signed_integral T>
T Negative(T value)
{
    return -value;
}

auto Increment(std::integral auto value)
{
    return value + 1;
}
```

작성 위치	특징
템플릿 선언 뒤	조건이 먼저 보임
함수 선언 뒤	긴 반환 타입에서 읽기 쉬움
매개변수 자리	간결한 제약 표현





```
template <typename T>
concept HasSize = requires(const T& value)
{
    value.size();
};
```

```
template <typename T>
concept HasValueType = requires
{
    typename T::value_type;
};
```

```
#include <concepts>
```

```
template <typename T>
concept ReadableHealth = requires(const T& value)
{
    { value.GetHp() } noexcept -> std::same_as<int>;
};
```

요구 조건	검사 내용
단순 요구	표현식이 유효한가
타입 요구	중첩 타입이 존재하는가
복합 요구	표현식, noexcept, 결과 타입
중첩 요구	추가 논리 조건

- **requires** 표현식은 실제 실행이 아니라 컴파일 시점 검사



10.6.5 사용자 정의 Concept

- Concept 만족 선언 문법은 없음
- 사용 지점에서 요구 조건 검사

```
#include <concepts>
```

```
template <typename T>  
concept DamageableObject =  
requires(T& target, int damage)  
{  
    { target.TakeDamage(damage) } ->  
        std::convertible_to<int>;  
    { target.IsDead() } ->  
        std::convertible_to<bool>;  
};  
  
template <DamageableObject T>  
int ApplyDamage(T& target, int damage)  
{  
    if (damage <= 0 || target.IsDead())  
    {  
        return 0;  
    }  
  
    return target.TakeDamage(damage);  
}
```

특징	설명
상속 필요 없음	요구 함수가 있으면 사용 가능
정적 인터페이스	컴파일 시점에 검사
이름 중요	DamageableObject처럼 역할 중심





- 타입 특성은 컴파일 시점에 타입 성질을 확인하거나 변환
- 〈type_traits〉 헤더 제공
- static_assert, if constexpr, Concept 정의에 사용

```
#include <type_traits>

static_assert(std::is_integral_v<int>);
static_assert(!std::is_integral_v<double>);

template <typename T> struct IsPointer
{
    static constexpr bool value = false;
};

template <typename T> struct IsPointer<T*>
{
    static constexpr bool value = true;
};

template <typename T>
inline constexpr bool IsPointerValue = IsPointer<T>::value;
```



```
#include <type_traits>
```

```
using First = std::remove_reference_t<int>;
```

```
using Second = std::remove_reference_t<const int&&>;
```

```
static_assert(std::is_same_v<First, int>);
```

```
static_assert(std::is_same_v<Second, const int>);
```

```
using Raw = std::remove_cvref_t<const int&>;
```

```
static_assert(std::is_same_v<Raw, int>);
```

```
using Pointer = std::add_pointer_t<int>;
```

```
using Pointee = std::remove_pointer_t<int*>;
```

```
using Common = std::common_type_t<int, double>;
```

```
static_assert(std::is_same_v<Common, double>);
```

변환	의미
<code>remove_reference_t</code>	참조 제거
<code>remove_cvref_t</code>	참조와 최상위 cv 제거
<code>decay_t</code>	값 전달과 유사한 변환
<code>common_type_t</code>	공통 타입 계산



- 컴파일 시점 조건문
- 조건은 상수 표현식
- 선택되지 않은 분기는 해당 특수화에서 인스턴스화되지 않음
- 하나의 구현 안에서 작은 타입별 차이를 처리할 때 적합

```
#include <iostream>
#include <type_traits>

template <typename T> void PrintCategory(const T& value)
{
    if constexpr (std::is_integral_v<T>)
    {
        std::cout << "integer: " << value << '\n';
    }
    else if constexpr (std::is_floating_point_v<T>)
    {
        std::cout << "real: " << value << '\n';
    }
    else
    {
        std::cout << "object: " << value << '\n';
    }
}
```





10.7.3 if constexpr와 requires

```
template <typename T>
void Clear(T& value)
{
    if constexpr (requires { value.clear(); })
    {
        value.clear();
    }
    else
    {
        value = T{};
    }
}
```

```
template <typename T>
auto GetValue(T value)
{
    if constexpr (std::is_pointer_v<T>)
    {
        return *value;
    }
    else
    {
        return value;
    }
}
```

상황	적합한 방법
함수 사용 가능 조건이 다름	Concept 제약 오버로딩
하나의 알고리즘 내부 일부만 다름	if constexpr
저장 구조와 공개 인터페이스가 다름	부분 특수화



- 스피네이(SFINAE): Substitution Failure Is Not An Error
- 템플릿 인수 대입 실패를 전체 오류가 아니라 후보 제거로 처리
- C++20 이전 제약 표현에 널리 사용

```
#include <type_traits>

template <typename T>
std::enable_if_t<std::is_integral_v<T>, T>
DoubleValue(T value)
{
    return value * 2;
}

int value = DoubleValue(10);
// double real = DoubleValue(1.5); // 후보 없음

template <std::integral T>
T DoubleValueModern(T value)
{
    return value * 2;
}
```

기존 방식

std::enable_if_t

조건이 타입 표현 안에 숨음

모던 C++20 방식

Concepts

선언에 요구 조건이 드러남



- C++20 신규 인터페이스는 Concepts 우선

기능	주된 역할	적합한 위치
타입 특성	타입 판정과 타입 변환	구현 내부, Concept 정의
<code>static_assert</code>	선택된 템플릿의 필수 조건 검사	클래스나 함수 본문
<code>if constexpr</code>	컴파일 시점 분기	함수와 멤버 함수 본문
SFINAE	부적합한 함수 후보 제거	C++17 호환 인터페이스
Concepts	요구 조건과 제약 오버로딩	C++20 공개 인터페이스
특수화	타입 형태에 따른 전체 구현 변경	클래스 저장 구조와 정책

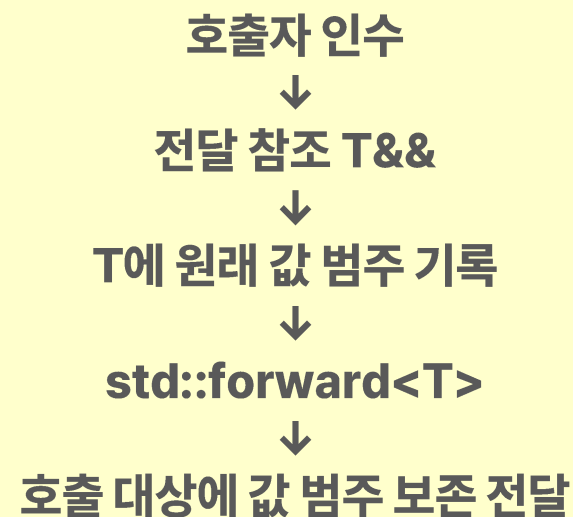
```
template <typename T>
concept HealthValue =
    std::integral<T> &&
    std::signed_integral<T>;
```

```
template <HealthValue T>
class Health
{
public:
    explicit Health(T maxHp)
        : hp(maxHp), maxHp(maxHp) {}
private:
    T hp;
    T maxHp;
};
```



- 제네릭 함수가 받은 인수를 다른 함수나 생성자에 전달
- 타입뿐 아니라 값 범주도 보존해야 할 수 있음
- lvalue은 lvalue으로, rvalue은 rvalue으로 전달
- 전달 참조와 `std::forward` 사용

도구	목적
참조 축약	겹친 참조 형태를 규칙에 따라 정리
전달 참조	lvalue과 rvalue 모두 수신
<code>std::forward</code>	원래 값 범주 복원
완벽 전달	생성자나 함수에 인수 중계





- rvalue 참조끼리 결합한 경우에만 rvalue 참조 유지
- lvalue 참조가 하나라도 포함되면 lvalue 참조

```
#include <type_traits>
```

```
using LValue = int&;  
using RValue = int&&;
```

```
using First = LValue&;  
using Second = LValue&&;  
using Third = RValue&;  
using Fourth = RValue&&;
```

```
static_assert(std::is_same_v<First, int&>);  
static_assert(std::is_same_v<Second,  
int&>);  
static_assert(std::is_same_v<Third, int&>);  
static_assert(std::is_same_v<Fourth,  
int&&>);
```

겹친 형태	축약 결과
T& &	T&
T& &&	T&
T&& &	T&
T&& &&	T&&



```
template <typename T>
void Relay(T&& value)
{
}
```

```
int number = 10;
const int limit = 20;
```

```
Relay(number);
Relay(limit);
Relay(10);
```

호출	인수 값 범주	T	매개변수 타입
Relay(number)	lvalue	int&	int&
Relay(limit)	상수 lvalue	const int&	const int&
Relay(10)	rvalue	int	int&&

```
template <typename T>
void ReceiveConstRvalue(const T&& value)
{
}
```

- **const T&&**는 전달 참조가 아님
- 정확히 추론되는 T&& 형태가 필요



10.8.3 std::forward

```
void Use(const std::string& value);
void Use(std::string&& value);
```

```
template <typename T>
void RelayWrong(T&& value)
{
    Use(value);
}
```

```
#include <utility>
```

```
template <typename T>
void RelayCorrect(T&& value)
{
    Use(std::forward<T>(value));
}
```

- 이름이 있는 변수 표현식은 선언 타입이 rvalue 참조 여도 lvalue
- RelayWrong(std::string("Mage"))도 lvalue처럼 전달

호출	전달 결과
RelayCorrect(name)	lvalue 유지
RelayCorrect(std::string("Mage"))	rvalue 유지

- std::forward<T>는 무조건 rvalue으로 바꾸는 함수가 아님



```
#include <memory>
#include <utility>

template <typename T, typename... Args>
std::unique_ptr<T> CreateObject(Args&&... args)
{
    return std::make_unique<T>(
        std::forward<Args>(args)...);
}

class Character
{
public:
    Character(std::string name, int hp)
        : name(std::move(name)), hp(hp)
    {
    }
private:
    std::string name;
    int hp;
};
```

```
std::string name = "Knight";
```

```
auto first = CreateObject<Character>(name, 100);
auto second =
    CreateObject<Character>(std::string("Mage"), 80);
```

- **Args&&...** 는 전달 참조 팩
- **std::forward<Args>(args)...** 로 각 인수의 값 범주 보존



10.8.5 std::move와 std::forward

항목	std::move	std::forward
목적	이동 가능한 표현식으로 변환	원래 값 범주 보존
주 사용 위치	일반 코드, 이동 연산 구현	전달 참조를 받는 제네릭 함수
결과	항상 rvalue 범주	T에 따라 lvalue 또는 rvalue
템플릿 인수	필요 없음	추론된 타입 인수 필요
사용 후 주의	원본이 이동된 상태 가능	rvalue 전달 시 이동 가능

```
std::string source = "Knight";  
std::string target = std::move(source);
```

```
template <typename T>  
void ForwardToUse(T&& value)  
{  
    Use(std::forward<T>(value));  
}
```

```
template <typename T>  
void RelayWithMove(T&& value)  
{  
    Use(std::move(value)); // lvalue도 이동 대상으로
```



10.8.6 객체지향과 제네릭의 결합

```
class GameObject
```

```
{  
public:  
    virtual ~GameObject() = default;  
    virtual void Update() = 0;  
    virtual std::string_view GetName() const noexcept = 0;  
};
```

```
template <typename T>
```

```
concept Damageable = requires(T& target, const T& constTarget, int damage)  
{  
    { target.TakeDamage(damage) } -> std::same_as<int>;  
    { constTarget.GetHp() } noexcept -> std::same_as<int>;  
    { constTarget.IsDead() } noexcept -> std::same_as<bool>;  
};
```

```
template <Damageable Target>
```

```
int ApplyDamage(Target& target, int damage)  
{  
    return target.TakeDamage(damage);  
}
```

위치	적용 방식
GameObject 계층	실행 시간 다형성
Damageable	정적 인터페이스
ApplyDamage()	컴파일 시점 제약



- 제네릭 프로그래밍은 타입 이름보다 필요한 연산과 의미에 의존
- 함수 템플릿은 함수 인수로부터 템플릿 인수 추론
- 값 매개변수는 최상위 const와 참조성 제거
- 참조 매개변수는 참조 대상의 한정자 보존
- 배열 참조는 배열 크기를 비타입 템플릿 인수로 추론 가능
- 템플릿은 사용된 인수 조합에 대해 인스턴스화
- 일반적인 템플릿 정의는 호출 지점에서 보여야 하므로 헤더에 둬
- 제한된 타입만 지원하면 명시적 인스턴스화로 분리 가능
- 클래스 템플릿의 각 템플릿 인수 조합은 서로 다른 구체 타입
- 별칭 템플릿은 새 타입이 아니라 타입 표현의 다른 이름





- 전체 특수화는 모든 템플릿 인수를 고정한 별도 구현
- 부분 특수화는 특정 타입 형태나 일부 인수 조건에 적용
- 함수 템플릿은 부분 특수화를 지원하지 않음
- 가변 인수 템플릿은 타입과 값의 개수가 정해지지 않은 팩을 받음
- 팩 확장은 문법적 패턴을 각 원소에 반복
- 폴드 표현식은 팩 원소 사이에 연산자를 결합
- Concepts는 템플릿 요구 조건을 선언에 표현
- requires 표현식은 타입과 표현식의 유효성을 검사
- 타입 특성은 컴파일 시점 타입 판정과 변환에 사용
- `if constexpr`는 선택된 템플릿 구현 안에서 타입별 코드 분기



- SFINAE와 enable_if는 기존 C++에서 후보 제거에 사용
- C++20 신규 인터페이스는 Concepts 우선 검토
- 전달 참조는 추론되는 T&& 매개변수
- 참조 축약으로 lvalue과 rvalue을 모두 받을 수 있음
- 이름이 있는 매개변수 표현식은 항상 lvalue
- std::forward<T>는 원래 값 범주를 복원
- std::move는 무조건 rvalue 범주로 변환
- 완벽 전달은 생성자나 함수에 인수를 중계할 때 사용
- 실행 시간 객체 종류 관리는 가상 함수가 적합
- 컴파일 시점 공통 연산은 템플릿과 Concept가 적합
- 실제 C++ 프로그램에서는 두 방식을 함께 사용

